

Manual de montaje y operacion del pipeline

Mortalidad y AVP por causa especifica

Para que sirve este manual

Este manual esta escrito para una persona con perfil informatico basico que necesita montar el proyecto por primera vez en una laptop, una PC Windows, un servidor o un contenedor.

La idea principal es muy simple:

- el proyecto puede leer el `raw` y los datasets externos desde cualquier ubicacion;
- los resultados se escriben dentro del repo;
- tu solo debes decirle al proyecto donde estan esos insumos.

No necesitas editar scripts metodologicos para cambiar rutas.

Dos formas de entrar al proyecto

Ruta 1. Trabajo clasico del analista

Si quieres correr el pipeline sin depender de un agente:

1. `README.md`
2. `docs/quickstart_first_use.md`
3. `docs/operations_manual.md`
4. este PDF solo si necesitas una guia mas extensa
5. `maestros/README_MAESTROS.md` cuando quieras entrar a la metodologia completa

Ruta 2. Trabajo asistido por agente Codex

Si quieres que un agente te ayude a entender, auditar o extender el proyecto:

1. `Agents.md`
2. `maestros/README_MAESTROS.md`
3. metodología 00-04
4. luego la ruta adicional segun la tarea

Segun tarea: - operar pipeline: manuales operativos - auditar o corregir: `maestros/agente/05_MASTER_PROMPT_A`
- crear tablas, figuras o manuscrito: `maestros/continuidad/08_MASTER_REPORTING_Y_PUBLICACION.txt`
- continuar trabajo reciente o abrir un hilo nuevo sobre main: `maestros/continuidad/09_MASTER_CONTEXTO_MIGR`
y `maestros/continuidad/10_MASTER_REGLA_TRABAJO_EN_MAIN.txt`

Ejemplo breve de prompt:

- “Lee `Agents.md`, el indice de maestros y la ruta de reporting; luego propon una bateria de figuras elegantes para publicacion.”

Regla general de resolucion de rutas

El proyecto busca las rutas en este orden:

1. variables de entorno
2. `config/runtime_paths.yml`
3. fallback local del repo

Esto significa lo siguiente:

- si defines una variable de entorno, esa variable tiene prioridad;
- si no defines variable, se usa `config/runtime_paths.yml`;
- si ambos estan vacios, el proyecto intenta usar el layout interno esperado del repo.

Antes de empezar

Antes de tocar ningun archivo, confirma estas cuatro cosas:

- ☐ ya se donde esta la carpeta de `sinadef`
- ☐ ya se donde estan los datasets externos principales
- ☐ ya decidi si usare variables de entorno o `runtime_paths.yml`
- ☐ tengo permiso de lectura sobre inputs y escritura sobre outputs

Paso 1. Decide que caso te corresponde

Usa esta tabla simple.

Caso	Donde esta <code>sinadef</code>	Donde estan los externos	Recomendacion
Caso A	Fuera del repo	Fuera del repo	Usar variables o <code>runtime_paths.yml</code>
Caso B	Dentro del repo	Fuera del repo	Dejar <code>sinadef</code> interno y configurar solo externos
Caso C	Dentro del repo	Dentro del repo	Dejar casi todo vacio y usar rutas relativas internas

Paso 2. Entiende que ruta necesita cada cosa

Que debe ser `MCE_SINADEF_DIR`

Debe apuntar a la **carpeta final** de SINADEF.

Ejemplos correctos:

- `D:/raw_protegido/sinadef`
- `/mnt/protected/raw/sinadef`
- `C:/proyectos/mortalidad-causa-especifica/data/raw/sinadef`

Ejemplos incorrectos:

- `D:/raw_protegido/sinadef/archivo_2022.csv`
- `C:/proyectos/mortalidad-causa-especifica/data/raw`
- `D:/`

Que debe ser `MCE_EXTERNAL_ROOT`

Debe apuntar a una raiz desde la cual tengan sentido los paths declarados en `config/external_sources.yml`.

Hoy los externos principales son:

- `population_result`
- `life_table_mortality_single_age`

- `life_table_standard_single_age`

En `config/external_sources.yml` veras rutas como estas:

- `../demografia-poblacion-inei/data/final/population_inei/population_result.parquet`
- `../tabla-mortalidad-peru/data/final/life_table_mortality/single_age/ref_life_table_morta`
- `../tabla-vida-estandar/data/final/standard_life_table/life_table_standard_reference_sing`

Si tu organizacion de carpetas es distinta, puedes usar overrides puntuales en `runtime_paths.yml`.

Paso 3. Como encontrar las rutas en disco

En Windows

1. Abre el Explorador.
2. Entra a la carpeta que quieres usar.
3. Haz clic en la barra de direccion.
4. Copia la ruta completa.
5. Pegala en PowerShell o en `runtime_paths.yml`.

Ejemplo practico:

- si abres `D:\repos_datos\demografia-poblacion-inei\data\final\population_inei`, esa es la ruta real de la carpeta.

En Linux o servidor

1. Abre la terminal.
2. Entra a la carpeta con `cd`.
3. Ejecuta `pwd`.
4. Copia la ruta completa.

Ejemplo:

```
cd /mnt/projects/demografia-poblacion-inei/data/final/population_inei
pwd
```

Paso 4. Entiende config/runtime_paths.yml

El archivo actual es este:

```
raw_root: ""
external_root: ""

inputs:
  sinadef_dir: ""

external_dataset_overrides:
  population_result: ""
  life_table_mortality_single_age: ""
  life_table_standard_single_age: ""
```

Que significa cada campo

raw_root

LLENAR si quieres apuntar a una carpeta madre que contiene **sinadef**.

DEJAR EN BLANCO si:

- usaras **MCE_SINADEF_DIR**, o
- **data/raw/sinadef/** ya vive dentro del repo.

external_root

LLENAR si quieres usar una raiz comun para los externos.

DEJAR EN BLANCO si:

- usaras **MCE_EXTERNAL_ROOT**, o
- todos tus externos ya viven dentro del repo, o
- usaras overrides puntuales.

inputs.sinadef_dir

LLENAR solo si quieres apuntar directamente a la carpeta final de **SINADEF**.

DEJAR EN BLANCO si **raw_root** ya resuelve **sinadef** correctamente o si usaras **MCE_SINADEF_DIR**.

`external_dataset_overrides.*`

LLENAR solo si un dataset externo esta en una ubicacion especial y no sigue el layout comun.

DEJAR EN BLANCO si `external_root` o `MCE_EXTERNAL_ROOT` ya resuelven bien la ruta.

Paso 5. Que llenar y que dejar en blanco segun tu caso

Caso A. Raw fuera del repo y externos fuera del repo

Este es el caso mas comun en servidor, Docker o instalaciones institucionales.

Opcion recomendada: variables de entorno

```
$env:MCE_SINADF_DIR="D:/raw_protegido/sinadef" # **LLENAR** con la carpeta final de sinadef
$env:MCE_EXTERNAL_ROOT="D:/repos_datos"      # **LLENAR** con la raiz comun de externos
```

En este caso, deja `config/runtime_paths.yml` completamente vacio:

```
raw_root: "" # **DEJAR EN BLANCO**
external_root: "" # **DEJAR EN BLANCO**

inputs:
  sinadef_dir: "" # **DEJAR EN BLANCO**

external_dataset_overrides:
  population_result: "" # **DEJAR EN BLANCO**
  life_table_mortality_single_age: "" # **DEJAR EN BLANCO**
  life_table_standard_single_age: "" # **DEJAR EN BLANCO**
```

Opcion alternativa: solo YAML

```
raw_root: "D:/raw_protegido" # **LLENAR** si `sinadef` vive bajo esta ruta
external_root: "D:/repos_datos" # **LLENAR** con la raiz comun de externos

inputs:
```

```

sinadev_dir: "" # **DEJAR EN BLANCO** si raw_root ya resue
external_dataset_overrides:
  population_result: "" # **DEJAR EN BLANCO**
  life_table_mortality_single_age: "" # **DEJAR EN BLANCO**
  life_table_standard_single_age: "" # **DEJAR EN BLANCO**

```

Caso B. Raw dentro del repo y externos fuera del repo

Este caso es comun en una PC de trabajo donde solo el raw principal esta embebido en el proyecto.

Supongamos esto:

- sinadev esta en data/raw/sinadev
- los externos viven en D:/repos_datos

Configuracion recomendada

```

$env:MCE_EXTERNAL_ROOT="D:/repos_datos" # **LLENAR**

```

Y deja runtime_paths.yml asi:

```

raw_root: "" # **DEJAR EN BLANCO** porque sinadev esta c
external_root: "" # **DEJAR EN BLANCO** si ya usas MCE_EXTER

inputs:
  sinadev_dir: "" # **DEJAR EN BLANCO**

external_dataset_overrides:
  population_result: "" # **DEJAR EN BLANCO**
  life_table_mortality_single_age: "" # **DEJAR EN BLANCO**
  life_table_standard_single_age: "" # **DEJAR EN BLANCO**

```

Caso C. Raw y externos dentro del mismo proyecto

Este caso es util para una laptop, una copia autocontenida o una prueba controlada.

Imagina este layout:

```
mortalidad-causa-especifica/
  data/
    raw/
      sinadef/
  externos/
    demografia-poblacion-inei/
    tabla-mortalidad-peru/
    tabla-vida-estandar/
```

Opcion practica con overrides puntuales

```
raw_root: "" # **DEJAR EN BLANCO**
external_root: "" # **DEJAR EN BLANCO**

inputs:
  sinadef_dir: "" # **DEJAR EN BLANCO** porque se usara data.

external_dataset_overrides:
  population_result: "externos/demografia-poblacion-inei/data/final/population_inei/population_inei.csv"
  life_table_mortality_single_age: "externos/tabla-mortalidad-peru/data/final/life_table_mortality_single_age.csv"
  life_table_standard_single_age: "externos/tabla-vida-estandar/data/final/standard_life_table.csv"
```

En este caso normalmente no necesitas variables de entorno.

Paso 6. Tres ejemplos completos y guiados

Escenario 1. Servidor / Linux / Docker

Situacion

- sinadef montado en /mnt/protected/raw/sinadef
- proyectos externos montados en /mnt/projects

Paso a paso

1. Verifica que existe /mnt/protected/raw/sinadef.
2. Verifica que existen los repos o carpetas externas bajo /mnt/projects.

3. Define variables de entorno.
4. Corre preflight.
5. Si el preflight pasa, corre el pipeline.

Comandos

```
export MCE_SINADF_DIR=/mnt/protected/raw/sinadef
export MCE_EXTERNAL_ROOT=/mnt/projects
Rscript ./scripts/run_preflight_checks.R
Rscript ./scripts/run_pipeline.R --profile full --clean-first
Rscript ./scripts/compare_validation_baseline.R
```

Que dejar en blanco

Si ya usaste esas variables, deja `runtime_paths.yml` vacio.

Escenario 2. PC Windows

Situacion

- sinadef en D:\raw_protegido\sinadef
- externos en D:\repos_datos

Paso a paso

1. Abre D:\raw_protegido\sinadef en el Explorador.
2. Confirma que esa carpeta es la correcta.
3. Abre D:\repos_datos y confirma que ahi viven los externos.
4. Copia ambas rutas desde la barra de direccion.
5. Abre PowerShell en la raiz del repo.
6. Define las variables.
7. Corre preflight.
8. Si pasa, corre el pipeline.

Comandos

```
$env:MCE_SINADef_DIR="D:/raw_protegido/sinadef"  
$env:MCE_EXTERNAL_ROOT="D:/repos_datos"  
Rscript .\scripts\run_preflight_checks.R  
Rscript .\scripts\run_pipeline.R --profile full --clean-first  
Rscript .\scripts\compare_validation_baseline.R
```

Variante con OneDrive o ruta larga

```
$env:MCE_SINADef_DIR="C:/Users/TuUsuario/OneDrive/Datos/sinadef"  
$env:MCE_EXTERNAL_ROOT="C:/Users/TuUsuario/OneDrive/ReposExternos"
```

Escenario 3. sinadef dentro del mismo proyecto

Situacion

- sinadef vive en data/raw/sinadef
- quieres que el proyecto sea autocontenido

Paso a paso

1. Confirma que existe data/raw/sinadef dentro del repo.
2. Decide si los externos tambien estaran dentro del repo.
3. Si los externos estaran dentro, define overrides puntuales en runtime_paths.yml.
4. Corre preflight.
5. Si pasa, corre el pipeline.

Comandos

```
Rscript .\scripts\run_preflight_checks.R  
Rscript .\scripts\run_pipeline.R --profile full --clean-first  
Rscript .\scripts\compare_validation_baseline.R
```

Que dejar en blanco

```
raw_root: "" # **DEJAR EN BLANCO**
external_root: "" # **DEJAR EN BLANCO** salvo que tengas una
inputs:
  sinadef_dir: "" # **DEJAR EN BLANCO**
```

Paso 7. Corre siempre el preflight primero

```
Rscript .\scripts\run_preflight_checks.R
```

Que revisar

- data/derived/qc/run_pipeline/preflight_checks.csv
- data/derived/qc/run_pipeline/preflight_summary.csv

Que significa que paso bien

Debes ver en ok al menos esto:

- runtime_input_sinadef_dir
- external_dataset_population_result
- external_dataset_life_table_mortality_single_age
- external_dataset_life_table_standard_single_age
- directorios escribibles de salida

Paso 8. Corre el pipeline completo

Cuando el preflight este aprobado:

```
Rscript .\scripts\run_pipeline.R --profile full --clean-first
```

Tambien puedes correr por perfil:

```
Rscript .\scripts\run_pipeline.R --profile core
Rscript .\scripts\run_pipeline.R --profile methods
Rscript .\scripts\run_pipeline.R --profile reports
```

Paso 9. Valida que todo termino bien

Despues del pipeline, corre:

```
Rscript .\scripts\compare_validation_baseline.R
```

Archivos a revisar

- data/derived/qc/run_pipeline/pipeline_run_log.csv
- data/derived/qc/baseline_compare/qc_baseline_vs_rerun_summary.csv

Que debes esperar

- todos los steps con `status = success`
- baseline sin diferencias sustantivas

Errores comunes y como corregirlos

Error 1. sinadef no existe

Sintoma: falla `runtime_input_sinadef_dir`.

Causa probable: apuntaste a la carpeta equivocada o a un archivo.

Como corregirlo: vuelve a ubicar la carpeta final de `sinadef` y corrige `MCE_SINADEF_DIR`, `raw_root` o `inputs.sinadef_dir`.

Error 2. Falta un externo

Sintoma: falla `external_dataset_population_result` o algun dataset requerido.

Causa probable: `MCE_EXTERNAL_ROOT` apunta demasiado arriba o demasiado abajo, o el layout no coincide con `config/external_sources.yml`.

Como corregirlo: revisa `config/external_sources.yml` y, si tu estructura no coincide, usa `external_dataset_overrides`.

Error 3. Llenaste a la vez `raw_root` y `sinadef_dir` con rutas distintas

Sintoma: confusion sobre cual ruta manda.

Causa probable: doble configuracion innecesaria.

Como corregirlo: usa solo una estrategia. Si ya usas `MCE_SINADEF_DIR`, deja vacios `raw_root` y `sinadef_dir`.

Error 4. Usaste rutas Linux en Windows o viceversa

Sintoma: el preflight no encuentra algo que si existe.

Causa probable: copiaste un ejemplo del sistema operativo incorrecto.

Como corregirlo: reemplaza la ruta por una del sistema real donde estas trabajando.

Error 5. Copiaste la ruta de un archivo cuando se esperaba una carpeta

Sintoma: el preflight sigue fallando.

Causa probable: usaste un `.csv`, `.parquet` o `.rds` donde se esperaba una carpeta.

Como corregirlo: usa carpeta para `MCE_SINADEF_DIR`, `raw_root` y `external_root`. Usa archivo individual solo en `external_dataset_overrides`.

Primer uso ultra guiado

Sigue este orden exacto:

1. localiza sinadef
2. localiza los externos principales
3. decide si tu caso es A, B o C
4. completa variables o `runtime_paths.yml`
5. corre `run_preflight_checks.R`
6. revisa `preflight_checks.csv`
7. corre `run_pipeline.R --profile full --clean-first`
8. corre `compare_validation_baseline.R`
9. revisa `pipeline_run_log.csv` y `qc_baseline_vs_rerun_summary.csv`

Comandos oficiales

```
Rscript .\scripts\run_preflight_checks.R
Rscript .\scripts\run_pipeline.R --profile full --clean-first
Rscript .\scripts\compare_validation_baseline.R
```

Semantica de carpetas del proyecto

Codigo

- `scripts/`: entrypoints operativos que un analista corre normalmente.
- `R/`: helpers y modulos compartidos.
- `R/diagnostics/`: diagnosticos reproducibles fuera del rerun normal.
- `R/maintenance/`: auditoria, gobernanza y mantenimiento no operativo diario.
- `reports/`: constructores de salidas editoriales y reportables.
- `config/`: configuracion y soporte.

Salidas

- `data/derived/qc/`: QC tabular canonico del pipeline.
- `reports/`: productos editoriales y reportables.
- `outputs/`: auxiliares no tabulares y no editoriales.

Politica operativa de warnings

Bloqueantes

- fallos de preflight
- steps del runner con `status != success`
- diferencias sustantivas en baseline

Tolerados pero documentados

- outliers de `y11_rate` en celdas extremas con denominadores muy pequenos
- fallback `parquet` -> `csv` del mismo dataset canonico cuando el reporte puede continuar sin cambiar de fuente metodologica

Registro vigente:

- `docs/minor_observations_status.csv`